

INFORME TÉCNICO

TaskFlow

Gestor de Tareas Web con API REST

Materia: Desarrollo Web / Programación

Institución: Universidad de Montemorelos

Integrante: Edmila Tchisseke Coelho da Cruz Camisa

Semestre: ISC 2° Semestre

Fecha: 6 de mayo de 2026

Práctica Integradora — Documento generado a partir del código fuente.

1. Introducción

El presente informe documenta el desarrollo de TaskFlow, una aplicación web de gestión de tareas construida como práctica integradora del segundo semestre de Ingeniería en Sistemas Computacionales. El proyecto aplica de forma conjunta los conceptos fundamentales del desarrollo web: HTML5 semántico, CSS3 con variables y layouts modernos, JavaScript del lado del cliente con Fetch API, y un servidor backend RESTful con Node.js y Express.

La aplicación permite a los usuarios crear, consultar, actualizar y eliminar tareas (operaciones CRUD completas), clasificarlas por categoría y prioridad, filtrarlas dinámicamente y marcarlas como completadas, todo ello mediante una interfaz responsiva y sin recargar la página gracias a la comunicación asíncrona con la API.

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación web full-stack que implemente un sistema completo de gestión de tareas, integrando un servidor RESTful con Express, una base de datos en memoria, y una interfaz de usuario dinámica con JavaScript vanilla que consuma la API mediante Fetch.

2.2 Objetivos Específicos

- Implementar una API REST con Express que exponga endpoints CRUD completos para la gestión de tareas.
- Diseñar una interfaz responsiva usando CSS Grid, Flexbox y variables CSS personalizadas.
- Consumir la API desde el frontend mediante Fetch API con async/await y manejo de errores.
- Aplicar HTML5 semántico con etiquetas header, nav, main, section, article y footer.
- Implementar filtrado de tareas por categoría mediante delegación de eventos en JavaScript.
- Validar formularios tanto desde HTML5 como desde JavaScript.
- Lograr diseño responsivo para dispositivos móviles mediante media queries.

3. Tecnologías Utilizadas

Tecnología	Versión	Rol en el Proyecto
Node.js	LTS	Entorno de ejecución JavaScript en el servidor
Express.js	^4.x	Framework para el servidor web y la API REST
JavaScript (Vanilla)	ES2020+	Lógica del cliente: Fetch, async/await, DOM
HTML5	Semántico	Estructura de la página con etiquetas semánticas
CSS3	Variables, Grid, Flex	Estilos, layout, animaciones y diseño responsivo
CORS (middleware)	npm	Permite peticiones cross-origin desde el navegador
Nodemon	dev	Reinicio automático del servidor en desarrollo

4. Arquitectura del Sistema

TaskFlow sigue una arquitectura cliente-servidor clásica de página única (SPA ligera). El servidor Express actúa como backend proporcionando la API REST y sirviendo los archivos estáticos del frontend. El cliente (navegador) consume dicha API de forma asíncrona sin recargar la página.

4.1 Backend — Servidor Express (server.js)

El servidor se configura con tres middlewares principales: `cors()` para permitir peticiones desde cualquier origen, `express.json()` para parsear cuerpos JSON, y `express.static('public')` para servir los archivos HTML, CSS y JS del frontend. La base de datos es un array en memoria (`tareas[]`) con un contador autoincremental (`nextId`) para los identificadores.

Campo	Tipo	Descripción
id	Number	Identificador único autoincremental
titulo	String	Nombre de la tarea (obligatorio, mín. 3 chars)
descripcion	String	Detalle opcional de la tarea
categoria	String	Clasificación: general, estudio, practica, proyecto, personal
prioridad	String	Nivel de urgencia: alta, media o baja
completada	Boolean	Estado de la tarea (false por defecto)
fechaCreacion	ISO String	Marca de tiempo de creación en formato ISO 8601

4.2 API REST — Endpoints

Método	Ruta	Código éxito	Descripción
GET	/api/tareas	200 OK	Obtener todas las tareas (filtro ?categoria=)
GET	/api/tareas/:id	200 OK	Obtener una tarea por ID
POST	/api/tareas	201 Created	Crear una nueva tarea
PUT	/api/tareas/:id	200 OK	Actualizar todos los campos de una tarea
PATCH	/api/tareas/:id/toggle	200 OK	Alternar estado completada/pendiente
DELETE	/api/tareas/:id	200 OK	Eliminar una tarea por ID

4.3 Frontend — Cliente JavaScript (app.js)

El cliente está organizado en tres capas funcionales dentro de `app.js`:

- **Capa de datos:** funciones `async` (`obtenerTareas`, `crearTarea`, `toggleTarea`, `eliminarTarea`) que encapsulan las llamadas `Fetch` a la API con manejo de errores `try/catch`.
- **Capa de renderizado:** `crearCardHTML` genera el HTML de cada tarjeta con `template literals`, y `renderizarTareas` actualiza el DOM y los contadores del header.

- **Capa de eventos:** manejadores del formulario, botones toggle/eliminar y filtros de categoría con delegación de eventos.

5. Estructura del Proyecto

Archivo	Descripción
server.js	Servidor Express, middlewares y endpoints de la API REST
public/index.html	Estructura HTML5 semántica de la aplicación
public/js/app.js	Lógica del cliente: Fetch API, DOM y eventos
public/css/style.css	Estilos con variables CSS, Grid, Flex y media queries
package.json	Metadatos del proyecto y scripts npm

6. Interfaz de Usuario

6.1 Estructura HTML5 Semántica

El documento HTML5 usa etiquetas semánticas correctamente. El header contiene la barra de navegación con el logo y los contadores globales. El main agrupa tres secciones: formulario de creación, barra de filtros y cuadrícula de tarjetas. El footer muestra la información del proyecto.

6.2 Formulario de Nueva Tarea

El formulario usa CSS Grid de dos columnas con campo de título (required, minlength=3), textarea de descripción, select de categoría y radio buttons personalizados para la prioridad. La validación opera en dos niveles: atributos HTML5 y verificación adicional en JavaScript con mensajes de error personalizados.

6.3 Tarjetas de Tarea

Cada tarea es un elemento article con borde izquierdo de color según prioridad (rojo=alta, amarillo=media, verde=baja). Las tareas completadas muestran el título tachado y opacidad reducida. Cada tarjeta incluye badges de categoría y prioridad, fecha formateada y botones de acción. Las tarjetas aparecen con animación fadeIn de 0.4 s.

6.4 Sistema de Filtros

Botones pill por categoría con delegación de eventos sobre el contenedor. Al seleccionar uno, envía el query param ?categoria= al servidor para filtrado desde el backend.

7. Diseño y Estilos CSS

7.1 Variables CSS

El archivo style.css define un sistema de diseño mediante variables en :root que cubren colores de marca, colores semánticos (éxito, peligro, advertencia), colores por prioridad, sombras, radio de borde y transiciones.

```
:root { --color-primario: #2563eb; --prioridad-alta: #ef4444;
  --prioridad-media: #f59e0b; --transicion: 0.3s ease; }
```

7.2 Layout y Responsivo

- **Grid:** formulario en 2 columnas y cuadrícula de tareas con repeat(auto-fill, minmax(320px, 1fr)).
- **Flexbox:** navegación, filtros, badges y acciones de tarjeta.
- **Media queries:** layout de 1 columna en móvil (<768 px) y ajuste de márgenes en pantallas pequeñas (<480 px).

8. Flujo de Datos

- **1. Acción del usuario:** interacción con el formulario, filtros o botones de tarea.
- **2. Fetch al servidor:** petición HTTP con método, ruta, cabeceras y cuerpo apropiados.
- **3. Procesamiento en Express:** validación, operación sobre el array y respuesta JSON estandarizada.
- **4. Actualización del DOM:** renderizarTareas() regenera la lista y actualiza contadores del header.

9. Instalación y Ejecución

1. Instalar dependencias:

```
npm install
```

2. Iniciar en modo desarrollo:

```
npm run dev
```

3. O en modo producción:

```
npm start
```

4. Abrir el navegador en <http://localhost:3000>

10. Resultados

A continuación se presentan capturas de pantalla de la aplicación TaskFlow en funcionamiento, mostrando las distintas funcionalidades implementadas.

10.1 Vista general de la aplicación

La siguiente captura muestra la interfaz principal con las dos tareas precargadas: 'Aprender HTML5 semántico' (pendiente, prioridad alta) y 'Practicar CSS Grid' (completada, con título tachado). Se aprecia el header con los contadores en tiempo real, el formulario de nueva tarea, los filtros de categoría y las tarjetas con sus badges de prioridad.

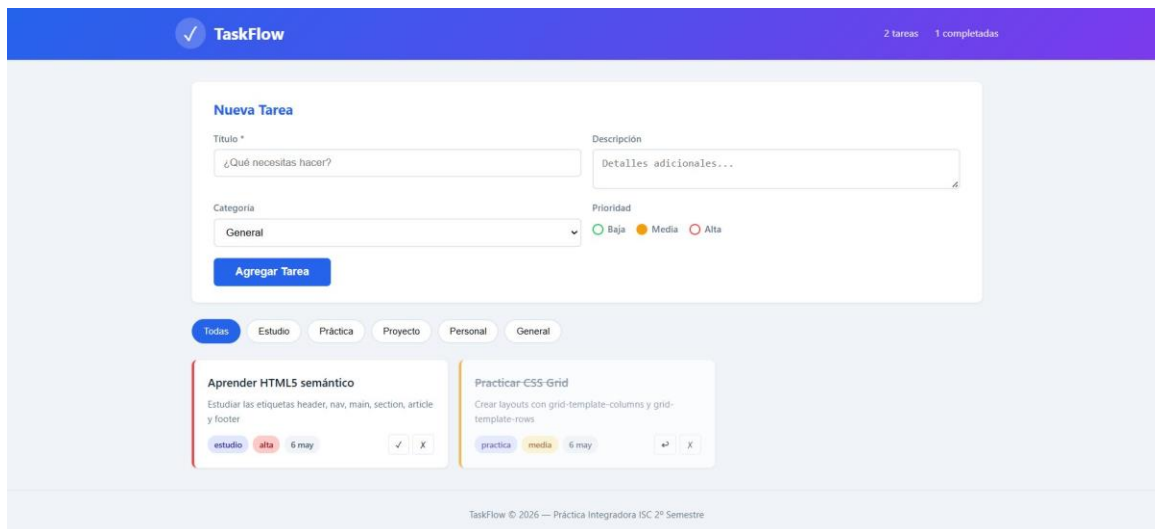


Figura 1. Vista general de TaskFlow con tareas activas y completadas.

10.2 Creación de una nueva tarea

La segunda captura muestra el resultado después de crear la tarea 'Subir proyectos a Github', con categoría 'estudio' y prioridad 'alta'. El contador del header se actualizó automáticamente a 1 tarea activa y el formulario se limpió tras el envío exitoso.

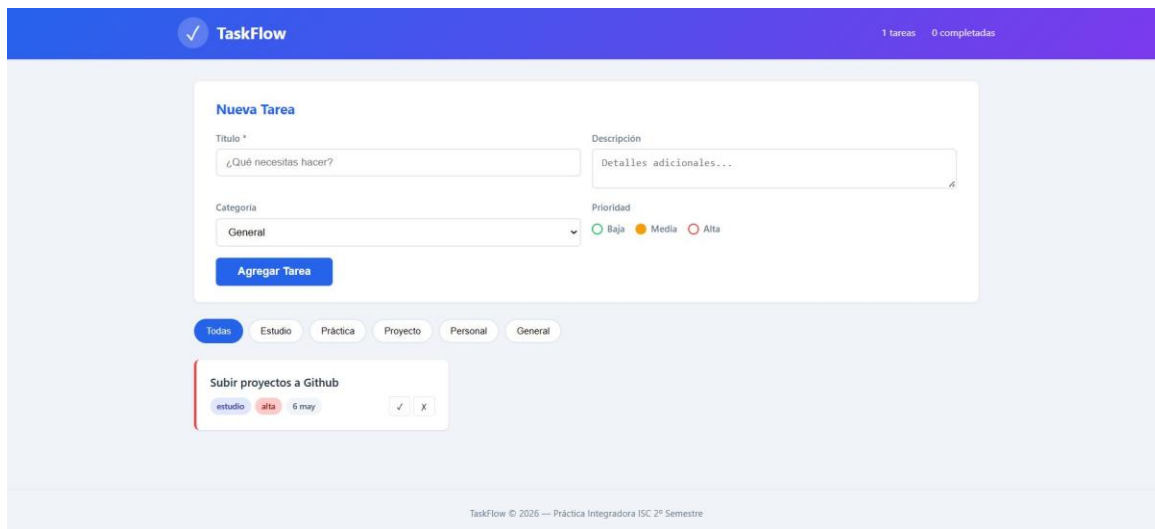


Figura 2. Nueva tarea 'Subir proyectos a Github' creada correctamente.

10.3 Marcar tarea como completada

La tercera captura muestra la misma tarea marcada como completada mediante el botón toggle (✓). El título aparece tachado, la tarjeta reduce su opacidad visualmente y el contador del header refleja '1 completadas' en tiempo real, sin necesidad de recargar la página.

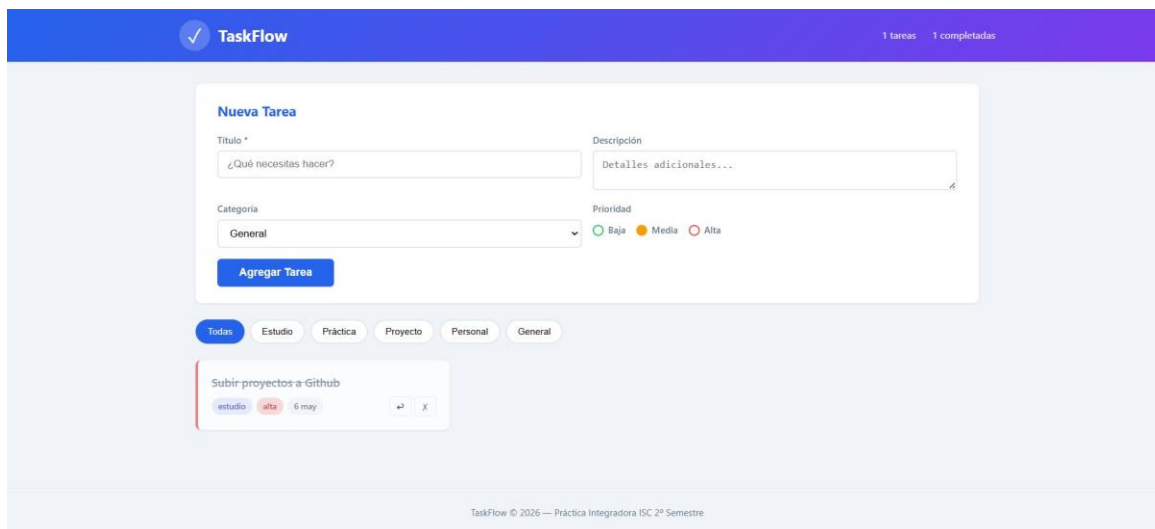


Figura 3. Tarea marcada como completada con actualización en tiempo real.

11. Conclusiones

TaskFlow integra de forma coherente todos los conceptos estudiados en el segundo semestre de ISC: HTML5 semántico, CSS3 con variables y layouts modernos, JavaScript asíncrono con Fetch API y Node.js con Express para un servidor RESTful con persistencia en memoria.

El proyecto demuestra la importancia de separar las capas de presentación, lógica y datos. La estandarización de las respuestas de la API facilita el mantenimiento del frontend y sienta las bases para una futura migración a base de datos real sin modificar el cliente.

Como trabajo futuro se propone: persistencia en base de datos (MongoDB o MySQL), autenticación de usuarios, edición en línea de tareas, ordenamiento por fecha o prioridad, y notificaciones para tareas con fecha límite.

12. Referencias

- MDN Web Docs. (2024). Fetch API. https://developer.mozilla.org/es/docs/Web/API/Fetch_API
- MDN Web Docs. (2024). CSS Custom Properties. https://developer.mozilla.org/es/docs/Web/CSS/--*
- MDN Web Docs. (2024). CSS Grid Layout. https://developer.mozilla.org/es/docs/Web/CSS/CSS_grid_layout
- Expressjs.com. (2024). Express — Fast, unopinionated web framework for Node.js. <https://expressjs.com>
- Node.js Foundation. (2024). Node.js Documentation. <https://nodejs.org/en/docs>